

Rapport de la SAE 1.01

Pecel Gabriel / Newerkowitsch Lucien / Sidhoum Sid-Ali

I-Fonctions et variables utilisées dans le programme :

Convention de notation des variables :

En Anglais

camelCase

Eviter les noms longs

A) Puissance 4 dans la console

- **createEmptyGrid()**

La fonction permet de créer une matrice vide grâce à laquelle on va créer la grille. La liste y (hauteur) contient toutes les listes x (largeur) et permet de gérer les cases grâce à leurs coordonnées y,x (dans cet ordre précis). Chaque case est représentée par un espace lorsqu'elle est vide.

```
def createEmptyGrid(width:int=7, height:int=6): #On crée une matrice
    return [[" " for x in range(width)] for y in range(height)]
```

- **showGrid()**

La fonction permet d'afficher la grille à partir de la matrice, qu'elle soit vide ou non, en séparant les cases d'une même ligne par un "|" et les lignes entre elles par une ligne de "-".

Variable	type	utilité
width	int	représenter la largeur de la grille
heigh	int	représenter la hauteur de la grille
grid	list	matrice passée en paramètres

```
def showGrid(grid:list): #à partir de la matrice on crée la grille
    width = len(grid[0])
    height = len(grid)
```

```

# on parcourt verticalement la grille pour avoir chaque ligne
print("\n 1  2  3  4  5  6  7") #on print le numéro des colonnes
print('-' * (width * 4 + 1)) # on print la ligne de séparation
for y in range(height):
    # on parcourt horizontalement la ligne pour avoir chaque case
    for x in range(width):
        print('| ', end='') # on print un pipe
        print(grid[y][x], end=' ') # on print le contenu de la case
    print('|') # une fois la ligne terminée, on print un pipe final
    print('-' * (width * 4 + 1)) # on print la ligne de séparation

```

```

grille_p4_v1.py x
1  def createEmptyGrill(width=7, height=6):
2      return [[' ' for x in range(width)] for y in range(height)]
3
4
5  def showGrill(grill):
6      width = len(grill[0])
7      height = len(grill)
8      # on parcourt verticalement la grille pour avoir chaque ligne
9      print('-' * (width * 4 + 1)) # on print la ligne de séparation
10     for y in range(height):
11         # on parcourt horizontalement la ligne pour avoir chaque case
12         for x in range(width):
13             print('| ', end='') # on print un pipe
14             print(grill[y][x], end=' ') # on print le contenu de la case
15         print('|') # une fois la ligne terminée, on print un pipe final
16         print('-' * (width * 4 + 1)) # on print la ligne de séparation
17
18
19  def main():
20      maGrille = createEmptyGrill()
21      maGrille[2][3] = 'O' # exemple de changement à la case x=3 y=2
22      showGrill(maGrille)
23
24
25  if __name__ == '__main__':
26      main()

```

```

Console x
-----
| | | | | | |
-----
| | | | | | |
-----
| | | O | | |
-----
| | | | | | |
-----
| | | | | | |
-----
| | | | | | |
-----

```

Un exemple de changement d'une des cases de la grille

- selectColumn()

La fonction permet de renvoyer la colonne choisie par l'utilisateur afin de jouer dans celle-ci, et elle permet d'éviter les erreurs de saisie, si on saisit autre chose qu'un entier, le programme ne crash pas, et nous demande de recommencer.

x	int (input)	renvoyer la colonne sélectionnée
---	-------------	----------------------------------

```
def selectColumn()->int: #Fonction qui sert à choisir les colonnes
    while True:
        try:
            x = int(input(" choisissez une colonne entre 1 et 7 >"))
            break
        except ValueError:
            print("Erreur de saisie, recommencez") #on empêche le programme de crash
            en cas de saisie incorrecte
            while 1>x or x>7:
                print("sélectionnez une colonne valide") #on vérifie que x est entre 1 et 7
            while True:
                try:
                    x = int(input("Choisissez une colonne entre 1 et 7 >"))
                    break
                except ValueError:
                    print("Erreur de saisie, recommencez")
            return x
#On demande à l'utilisateur de saisir la colonne où il veut jouer
```

- **playerMove()**

La fonction permet de prendre en compte le coup joué avec selectColumn() par l'utilisateur en fonction de si c'est le jaune ou le rouge, et permet de remplacer la case vide par 'J' ou par 'R'.

x	int	savoir la colonne jouée
y	int	utilisé pour vérifier la ligne
b	int	savoir de quel joueur c'est le tour
p4Game	list	Matrice de la grille

```
def playerMove(p4Game:list,b:int)->list: #action du joueur
    x=selectColumn()
    x-=1 #on sélectionne la colonne choisie par l'utilisateur (x allant de 0 à 6 il faut retirer 1 pour choisir la bonne colonne)
```

```

y=5 #on définit y pour placer le pion dans la dernière liste de la matrice
while p4Game[0][x] != " ":
    print("La colonne est pleine, choisissez une autre colonne") #On empêche de
jouer une colonne pleine
    x=selectColumn()
    x-=1
while p4Game[y][x] != " " or y == 0:
    y=y-1 #On remonte la colonne jusqu'à trouver une case vide
if p4Game[y][x] == " " :
    if b==0:
        p4Game[y][x] = 'J' #on print J dans la case vide
    elif b==1 :
        p4Game[y][x] = 'R' #on print R dans la case vide
    showGrid(p4Game) #on actualise la grille avec le coup joué
return p4Game

```

- playerName()

La fonction permet de saisir le nom des joueurs

x	int	nom du joueur
---	-----	---------------

```

def playerName()->str:
    x=input(" sélectionnez votre pseudonyme > ")
    return x

```

```

| | | | | | |
|-----|
joueur jaune sélectionnez votre pseudonyme > titi
joueur rouge sélectionnez votre pseudonyme > toto
titi choisissez une colonne entre 1 et 7 >1
Erreur de saisie, recommencez
choisissez une colonne entre 1 et 7 >1

```

```

| 0 | | | | | |
|-----|
toto choisissez une colonne entre 1 et 7 >8
sélectionnez une colonne valide
Choisissez une colonne entre 1 et 7 >7

1 2 3 4 5 6 7

```

on sélectionne le pseudonyme des joueurs (ici “titi” et “toto”) et la fonction va préciser le nom du joueur en lui demandant la saisie, elle va aussi empêcher une erreur de saisie ou un nombre incorrect

```

    1   2   3   4   5   6   7
-----
| J |   |   |   |   |   |   |
-----
| R |   |   |   |   |   |   |
-----
| J |   |   |   |   |   |   |
-----
| J |   |   |   |   |   |   |
-----
| R |   |   |   |   |   |   |
-----
| J |   |   |   |   |   | R |
-----
toto choisissez une colonne entre 1 et 7 >1
La colonne est pleine, choisissez une autre colonne
choisissez une colonne entre 1 et 7 >

```

La fonction **playerMove()** renvoie également si la colonne est pleine

- **winGame()**

La fonction permet de vérifier les condition de victoire en vérifiant ligne par ligne, colonne par colonne, et diagonale par diagonale, même si ça n'est pas le plus optimisé, nous n'avions pas le temps de faire mieux

lines	int	savoir le nombre de lignes
columns	int	savoir le nombre de colonnes

```
def winGame(p4Game:list)->bool:
```

```
    lines = 6
```

```
    columns= 7
```

```
    #vérification verticale
```

```
    for col in range(columns):
```

```
        for lin in range(lines-3):
```

```
            if p4Game[lin][col] == p4Game[lin+1][col] == p4Game[lin+2][col] ==
p4Game[lin+3][col] != ' ':

```

```
                return True
```

```
    # vérification Horizontale
```

```
    for lin in range(lines):
```

```
        for col in range(columns-3):
```

```

        if p4Game[lin][col] == p4Game[lin][col+1] == p4Game[lin][col+2] ==
p4Game[lin][col+3] != ' ':
            return True

# vérification première diagonale
for lin in range(lines-3):
    for col in range(columns-3):
        if p4Game[lin][col] == p4Game[lin+1][col+1] == p4Game[lin+2][col+2] ==
p4Game[lin+3][col+3] != ' ':
            return True

#autre diagonale
for lin in range(3, lines):
    for col in range(columns-3):
        if p4Game[lin][col] == p4Game[lin-1][col+1] == p4Game[lin-2][col+2] ==
p4Game[lin-3][col+3] != ' ':
            return True

# Pas de vainqueur
return False

```

- turnOrder()

La fonction permet de lancer le jeu, appelle playerName() pour entrer le nom des joueurs, ensuite elle fait jouer tour par tour les joueurs en appelant playerMove() et vérifie en appelant winGame si un des joueurs gagne. Enfin elle permet de vérifier si la grille est remplie et qu'il y a match nul.

x	int	déterminer qui joue
endGame	bool	savoir si la grille est pleine
winCondition	bool	déterminer si quelqu'un a gagné
n1 / n2	str	noms des joueurs

```

def turnOrder(p4Game:list): #fonction pour les tours de jeu
    endGame: bool
    x:int
    winCondition: bool
    print("joueur jaune",end="")

```

```

n1=playerName()
print("joueur rouge",end="")
n2=playerName()
endGame=False #On définit un bool qui va nous servir en cas de match null
winCondition=False #bool qui va servir à savoir si un joueur a gagné
while not endGame and not winCondition: #Tant que la grille n'est pas pleine et
qu'aucun joueur n'a gagné on fait jouer successivement les deux joueurs
    x=0
    print(n1,end= "")
    p4Game=playerMove(p4Game,x)
    winCondition=winGame(p4Game)
    if not winCondition: #si le joueur jaune n'a pas gagné
        x=1
        print(n2,end= "")
        p4Game=playerMove(p4Game,x)
        winCondition=winGame(p4Game)
    endGame= not(p4Game[0][0] == ' ' or p4Game[0][1] == ' ' or p4Game[0][2] == ' '
or p4Game[0][3] == ' ' or p4Game[0][4] == ' ' or p4Game[0][5] == ' ' or p4Game[0][6]
== ' ')
    #on vérifie que la grille n'est pas remplie
    if winCondition and x==0:
        print(n1," a gagné! bravo!")
    elif winCondition and x==1:
        print(n2," a gagné! bravo!") #On vérifie qui a gagné pour l'afficher
    elif endGame and not winCondition:
        print("la grille est remplie, match nul")

```

The left screenshot shows a 7x7 grid with columns numbered 1 to 7. The board is partially filled with 'R' (Red) and 'J' (Yellow) pieces. The right screenshot shows the same 7x7 grid, but it is now completely full. At the bottom of the grid, the text "la grille est remplie, match nul" is displayed.

la fonction winGame et la fonction turnOrder renvoient la personne qui a gagné
turnOrder permet également de définir l'égalité (pour l'exemple on utilise une version
sans condition de victoire car c'est plus simple à illustrer).

- main()

La fonction main va lancer successivement l'affichage de la grille vide puis la
fonction turnOrder pour démarrer la partie

```
def main():
#   maGrille = createEmptyGrill() #(grille de test)
#   maGrille[5][4] = 'J' # exemple de changement à la case x=4 y=5
#   showGrill(maGrille)
    p4Game = createEmptyGrid()
    showGrid(p4Game) # On définit la grille puis on lance la partie
    turnOrder(p4Game) # On lance la partie
```

B) Puissance 4 dans Pygame:

Fonction pour le puissance 4 :

- drawGrid():

La fonction permet d'afficher une grille pour le jeu à partir d'une matrice , à partir de cela nous avons configuré la taille de la grille , puis le nombre de colonnes et de lignes nécessaire au Puissance 4 . Ensuite on a créé trois boucles: une pour dessiner les colonnes , une pour dessiner les lignes et une autre pour dessiner les cercles à l'intérieur de la grille . Puis on a créé les numéros de colonne en dessous de la grille pour que chaque numéros corresponde avec la grille .

```
# Fonction pour dessiner la grille :
def drawGrid():
    #la couleur de la grille
    column_rect = pygame.Rect(200, 100, 900, 700)
    pygame.draw.rect(windows, color.Blue, column_rect)
    sizeNb = pygame.font.SysFont("arial", 45)
    pygame.display.update()
    nbColumn = 7
    nbLines = 6

    # Création de la grille vide avec les cercles :
    y=150
    for i in range (0,nbLines):
        x=250
        for j in range(0,nbColumn):
            pygame.draw.circle(windows,(color.Grey),(x,y),(48),50)
            x+=100
        y+=100
    #En dessous de la grille (numéro de colonnes)
    column_rect = pygame.Rect(200, 700, 900, 800)
    pygame.draw.rect(windows, (color.PastelBlue), column_rect)
    sizeNb = pygame.font.SysFont("arial", 45)
```



```
column = sizeNb.render("1",1,(color.DarkSilver))
windows.blit(column,[240,720])
```

```
column = sizeNb.render("2",1,(color.DarkSilver))
windows.blit(column,[340,720])
```

```
column = sizeNb.render("3",1,(color.DarkSilver))
windows.blit(column,[440,720])
```

```
column = sizeNb.render("4",1,(color.DarkSilver))
windows.blit(column,[540,720])
```

```
column = sizeNb.render("5",1,(color.DarkSilver))
windows.blit(column,[640,720])
```

```
column = sizeNb.render("6",1,(color.DarkSilver))
windows.blit(column,[740,720])
```

```
column = sizeNb.render("7",1,(color.DarkSilver))
windows.blit(column,[840,720])
```

```
# création de la grille avec les lignes et les colonnes :
x = 200
for i in range(0,8):
    pygame.draw.line(windows, (color.DarkBlue), (x, 100), (x, 700), 3)
    x += 100
pygame.display.update()
```

```
y=100
for j in range(0,8):
    pygame.draw.line(windows, (color.DarkBlue),(200,y),(900,y),4)
    y+=100
    pygame.display.update()
```

- placePawn():

Cette fonction permet de placer les pions dans la grille. Pour cela nous avons placé en paramètres le nombre de lignes et la taille de la case souhaitée , pour on a créé une boucle pour placer les pions par rapport aux lignes , on commence de la ligne la plus basse jusqu'à la ligne la plus haute . .

```
# Fonction qui permet de placer les pions dans la grille :
def placePawn(column ,color):
    nbLines=6
    sizeCase=100
    for line in range (nbLines -1,-1,-1):
```

```

        if grid[line][column] is None :
            x=250+column*sizeCase
            y=150+line*sizeCase
            pygame.draw.circle(windows,color,(x,y),48)
            grid[line][column]=color
        return True
    return False

```

-gridFull():

Cette fonction permet de détecter quand la grille est pleine. Pour cela nous avons créé une boucle qui renvoie une valeur booléenne , si la grille est vide elle renvoie False sinon elle renvoie True car la grille est pleine .

```

def gridFull ():
    for line in grid:
        if None in line: # pour cette ligne si les lignes de la grille ne sont pas remplies
            alors
                return False
    return True

```

- winCondition():

La fonction permet de vérifier une condition de victoire c'est-à-dire si 4 pions de couleur rouge ou jaune sont alignés horizontalement, verticalement, diagonalement de gauche à droite et de droite à gauche . Cette fonction est constituée de 4 boucles permettant de vérifier ces 4 conditions.

```

# Condition de victoire horizontal :
def winCondition (player):
    for line in grid:
        for i in range(4):
            if all(j == player for j in line[i:i+4]):
                return True

    for j in range(7):
        for i in range(3):
            if all(grid[i+k][j] == player for k in range (4)):
                return True

    for i in range(6):
        for j in range (7):
            if i+3<6 and j+3<7:
                if all(grid[i+k][j+k] == player for k in range (4)):

```

```
return True
```

```
for i in range(6):
```

```
    for j in range (7):
```

```
        if i+3<6 and j-3>=0:
```

```
            if all(grid[i+k][j-k] == player for k in range (4)):
```

```
                return True
```

```
return False
```

- Main():

Le Main permet de lancer le jeu. Il va lancer drawGrid pour dessiner la grille puis définir les variables , pour ensuite lancer la boucle pour le jeu .

```
# Fonction Principal qui lance les fonctions :
```

```
def main():
```

```
    drawGrid()
```

```
    exitClick = False
```

```
    running = True
```

```
    player=color.Yellow
```

```
    gameOver = False
```

```
    currentPlayer = gameColor.Yellow
```

```
    pygame.display.update()
```

C) Variables Puissance 4 dans Pygame:

1) Variable dans la saisie des pseudos:

Nom des variables	Type de variable	Fonction
windows	Tuple (int, int)	Génère la fenêtre Pygame
myfont	str et int	Police de caractère pour le texte et la taille de la police
image	N/A	Image “start” chargée
image2	N/A	Image “exit” chargée
exitClick	bool	Un indicateur de clic sur l'image ‘exit’

player1	str	Chaine de caractère vide pour stocker le premier pseudo
player2	str	Chaine de caractère vide pour stocker le deuxième pseudo
inputRect	Tuple (int ,int ,int , int)	Rectangle pour la saisie du pseudo du joueur 1
inputRect2	Tuple (int ,int ,int , int)	Rectangle pour la saisie du pseudo du joueur 2
colorPassiv	Couleur(Tuple)	Couleurs de base pour les rectangles inactifs
color	Couleur(Tuple)	Couleur du rectangle de saisie du pseudo 1
color2	Couleur(Tuple)	Couleur du rectangle de saisie du pseudo 2
lab	Rendu du texte(pygame.Surface)	Texte avec la police et la taille de 'myfont'
lab2	Rendu du texte (pygame.Surface)	Texte avec la police et la taille de 'myfont'
activ	bool	Indicateur d'activation de la saisie du pseudo 1
activ2	bool	Indicateur d'activation de la saisie du pseudo 2
player1Final	bool	Indicateur de finalisation de la saisie du pseudo 1
player2Final	bool	Indicateur de finalisation de la saisie du pseudo 2
running	bool	Indicateur pour maintenir la boucle principal actif
inputRect.w	Dynamique	La largeur du rectangle s'ajuste en fonction du pseudo 1
inputRect2.w	Dynamique	La largeur du rectangle s'ajuste en fonction du pseudo 2

textSurface	Rendu de texte (pygame.Surface)	Texte saisi pour le joueur 1 avec le rendu 'myFont'
textSurface2	Rendu de texte (pygame.Surface)	Texte saisi pour le joueur 2 avec le rendu 'myFont'

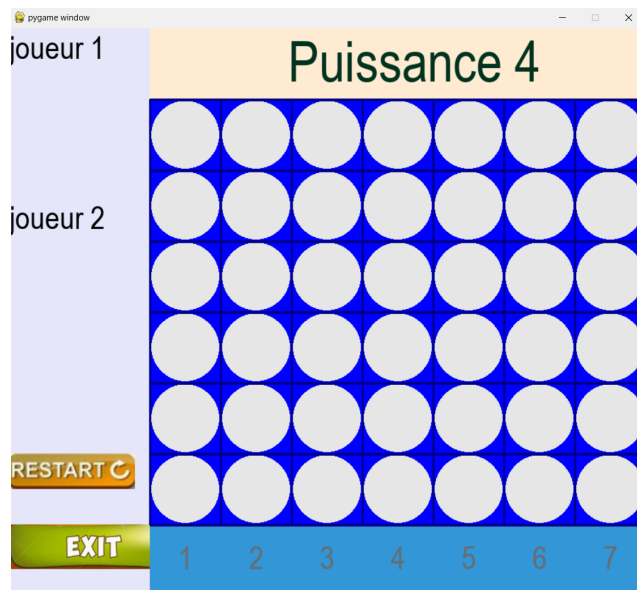
2) Variables dans le jeu Puissance 4 :

<u>Noms des variables</u>	<u>Type de variable</u>	<u>Fonction</u>
windows	pygame.Surface	Fenêtre du jeu
myFont	pygame.font.Font	Police de caractère
image	pygame.Surface	Image 'restart'
image2	pygame.Surface	image 'exit'
columnRect(2à6)	pygame.Rect	Rectangle avec le fond beige
lab	pygame.Surface	Texte "Puissance 4"
sizeNb	pygame.font.Font	Police de caractère pour les numéros de la colonnes
column	pygame.Surface	Texte pour les numéros de colonne
player1	str	Pseudo
player2	str	Pseudo
textRect	pygame.Rect	Rectangle pour le texte de la colonne d'affichage
grid	matrice	Grille du jeu
nbColumn	int	Nombre de colonnes
nbLines	int	Nombre de lignes
y	int	variable utilisée pour boucler à travers les colonnes
x	int	variable utilisée pour boucler à travers les lignes

sizeCase	int	Taille de la case de la grille
line	int	Variable utilisée pour boucler à travers les lignes de la grille.
currentPlayer	Tuple(couleur)	Couleur du joueur(rouge ou jaune)
exitClick	bool	Indicateur de clic sur l'image 'exit'
running	bool	Indicateur pour maintenir la boucle principale actif
player	Tuple (couleur)	Couleur du joueur actuel
imageWidth , imageHeight	int	Largeur et Hauteur de l'image (image et image2)
gameOver	bool	Indicateur pour gérer la fin
textMessage	pygame.Surface	Message pour afficher la condition de victoire
sizeFont	pygame.font.Font	Police de caractère pour les message de victoire et d'égalité

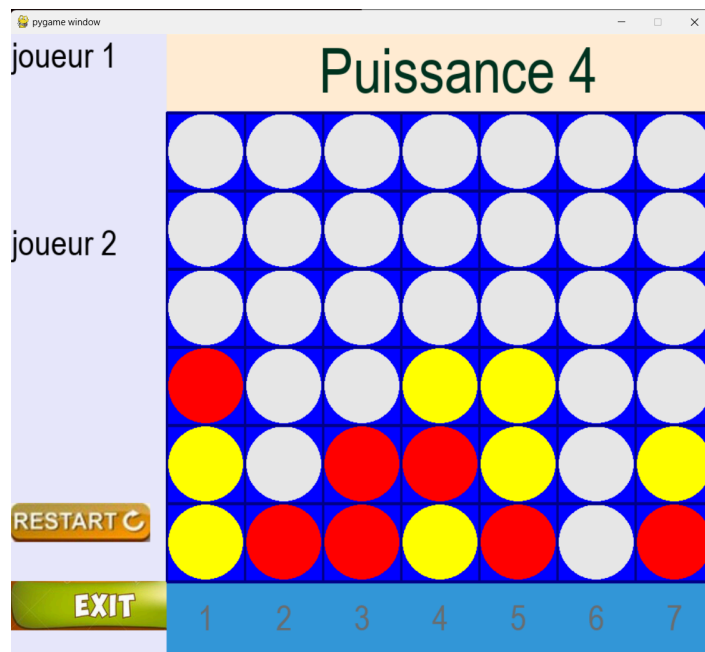
D) Test des fonctions Puissance 4 :

drawGrid() :



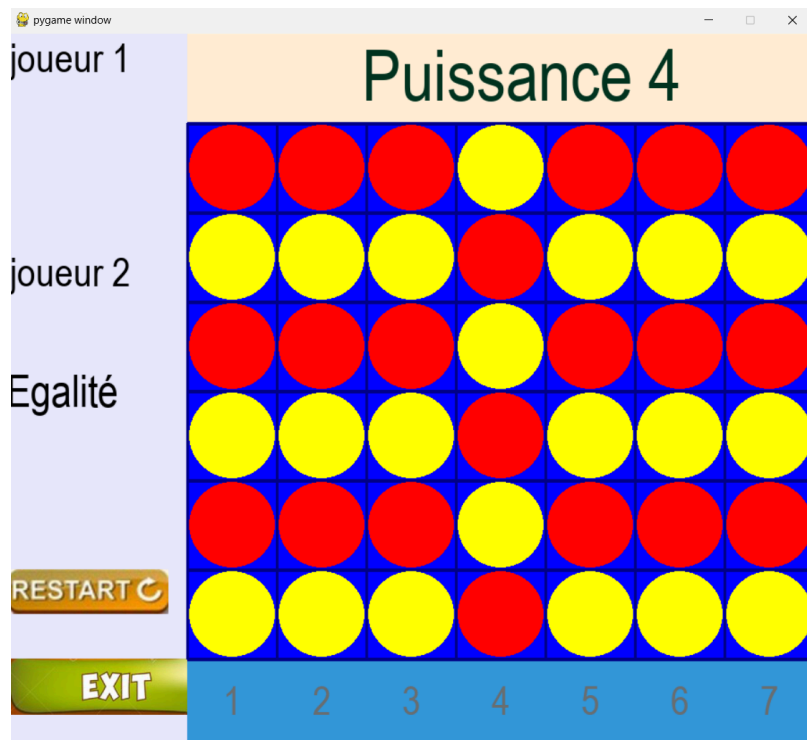
placePawn():

On a testé en plaçant quelques pions pour savoir si on pouvait les placer dans la grille



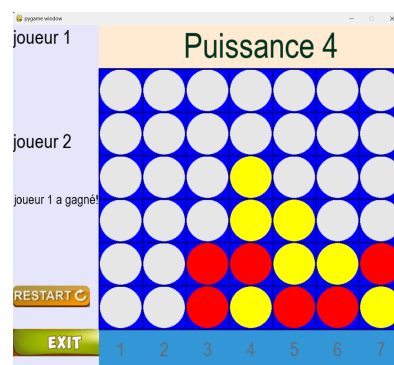
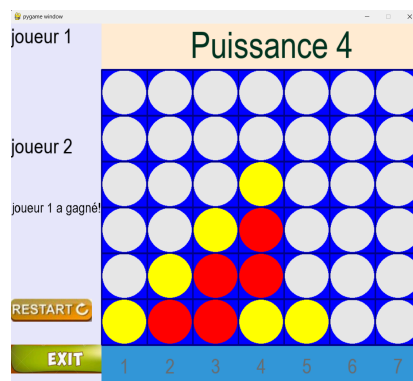
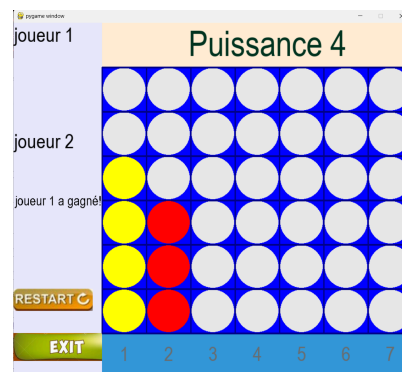
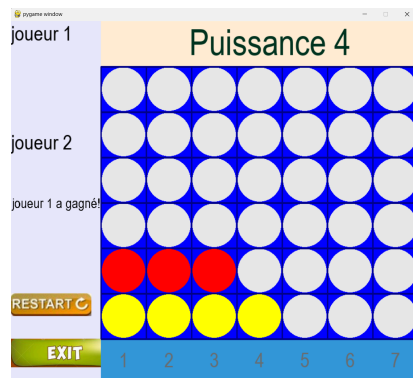
gridFull():

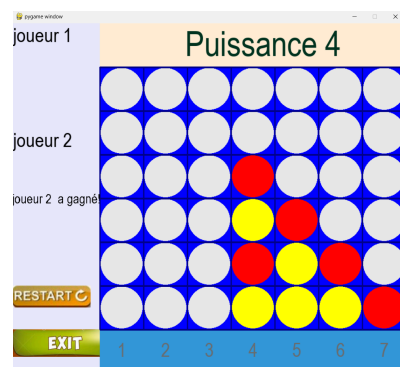
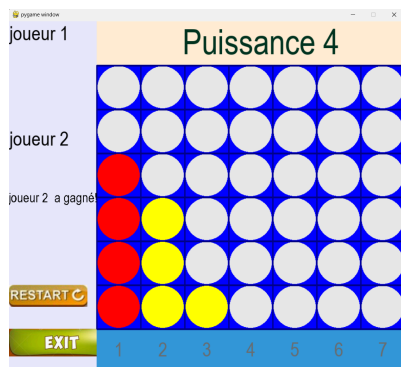
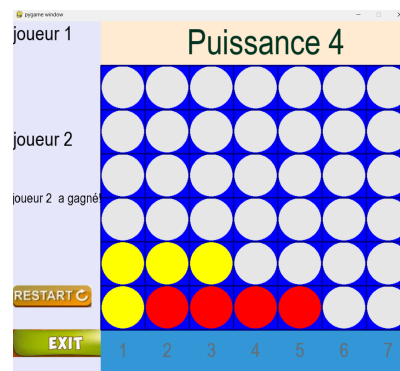
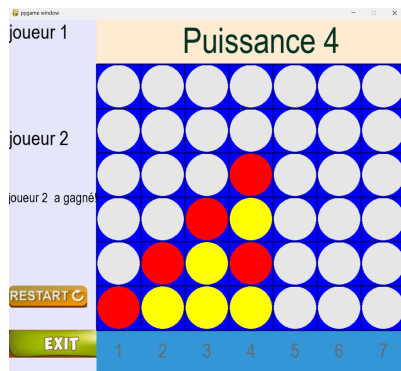
On a rempli la grille pour voir si la fonction pour voir l'égalité si la grille est pleine



winCondition():

Pour ce test nous avons testé chaque condition de victoire pour savoir si les fonctions fonctionnent



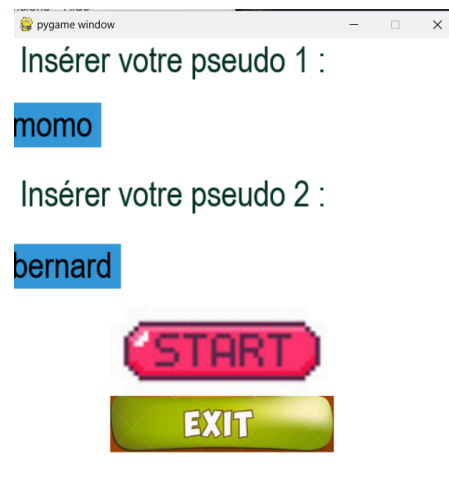
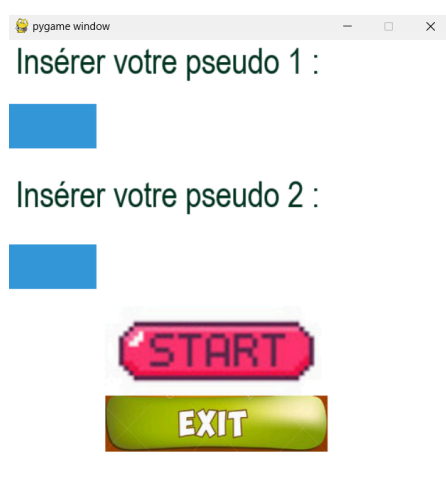


main():

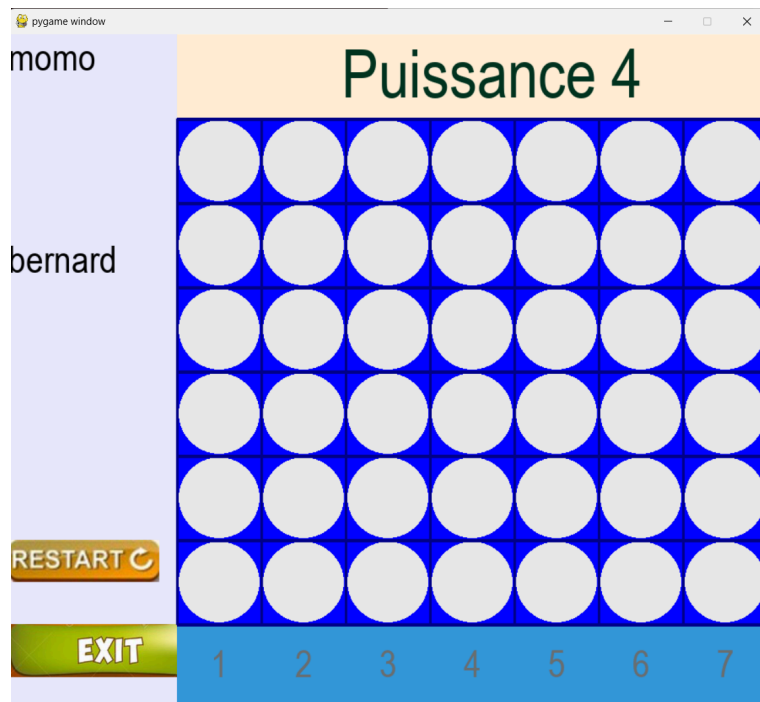
Pour cette fonction, ça revient à toutes les captures d'écran.

E) Test des fonctions de saisie :

Pour ce programme nous avons testé en saisissant des pseudo dans la fenêtre dédié à cela



Puis nous voyons que les pseudo s'affiche dans la grille



II- Répartition des tâches :

Gabriel : Programme dans la console + Aide sur pygame

Lucien : Programme pygame + un peu d'aide pour le programme console

Sid-Ali : ∅ (A fait des recherches toute la saé mais n'a visiblement rien trouvé, a codé 10 lignes de codes, qui ne fonctionnent pas, sur feuille A4!!!)